

Project - High Level Design on Chat With Energy Documents(RAG System)

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Ansh Upadhyay	EN22CS301147
02	Anuroop Patidar	EN22CS301176
03	Arnika Jain	EN22CS301196
04	Arnoday Soni	EN22CS301197
05	Aryan Singh	EN22CS301214

Group Name: Group 01D2

Project Number: GAI – 13

Industry Mentor Name:

University Mentor Name: Dr. Hemlata Patel Mam

Academic Year: 2026

Table Of Content

1. Introduction.....	3
1.1 Scope of the Document.....	3
1.2 Intended Audience	3
1.3 System Overview	3
2. System Design.....	4
2.1 Application Design.....	4
2.2 Process Flow	5
2.3 Information Flow	6
2.4 Components Design	7
2.5 Key Design Considerations.....	7
2.6 API Catalogue	8
3. Data Design.....	8
3.1 Data Model.....	8
3.2 Data Access Mechanism	10
3.3 Data Retention Policies.....	10
3.4 Data Migration	10
4. Interfaces	10
5. State and Session Management	10
6. Caching	11
7. Non-Functional Requirements.....	12
7.1 Security Aspects.....	12
7.2 Performance Aspects.....	12
8. References.....	12

1. Introduction

1.1 Scope of the Document

This document describes the High-Level Design (HLD) of the “Chat with Energy Documents” system, a Retrieval-Augmented Generation (RAG) based document intelligence solution tailored specifically for Energy domain documents.

The system enables users to upload professional Energy-related PDFs such as plant operation manuals, turbine reports, grid system documents, and technical specifications. After processing these documents, users can interact with them using natural language questions, and the system generates responses strictly grounded in the uploaded content.

This document covers the overall system architecture, application design, data handling mechanisms, API structure, non-functional requirements, and integration points. It focuses on architectural and structural decisions rather than implementation-level coding details.

1.2 Intended Audience

This document is intended for:

- Project evaluators and instructors
- Backend developers
- AI/ML engineers
- System architects
- DevOps engineers

It provides a comprehensive understanding of how the system is structured and how different modules interact.

1.3 System Overview

The “Chat with Energy Documents” system is designed using a Retrieval-Augmented Generation (RAG) architecture. Instead of fine-tuning a language model, the system retrieves relevant information from uploaded documents and provides that information to a Large Language Model (LLM) to generate context-aware responses.

The system operates in two main phases:

1. Document Ingestion and Indexing
2. Context-Aware Question Answering

During document ingestion, uploaded PDFs are processed, segmented into overlapping chunks, converted into vector embeddings, and stored in a vector database. During query processing, the user’s question is converted into an embedding, compared against stored embeddings using similarity search, and the most relevant chunks are retrieved. These chunks are passed to the LLM to generate a response that is strictly based on document content.

The system integrates embedding and LLM services from OpenAI and uses FAISS for vector similarity search.

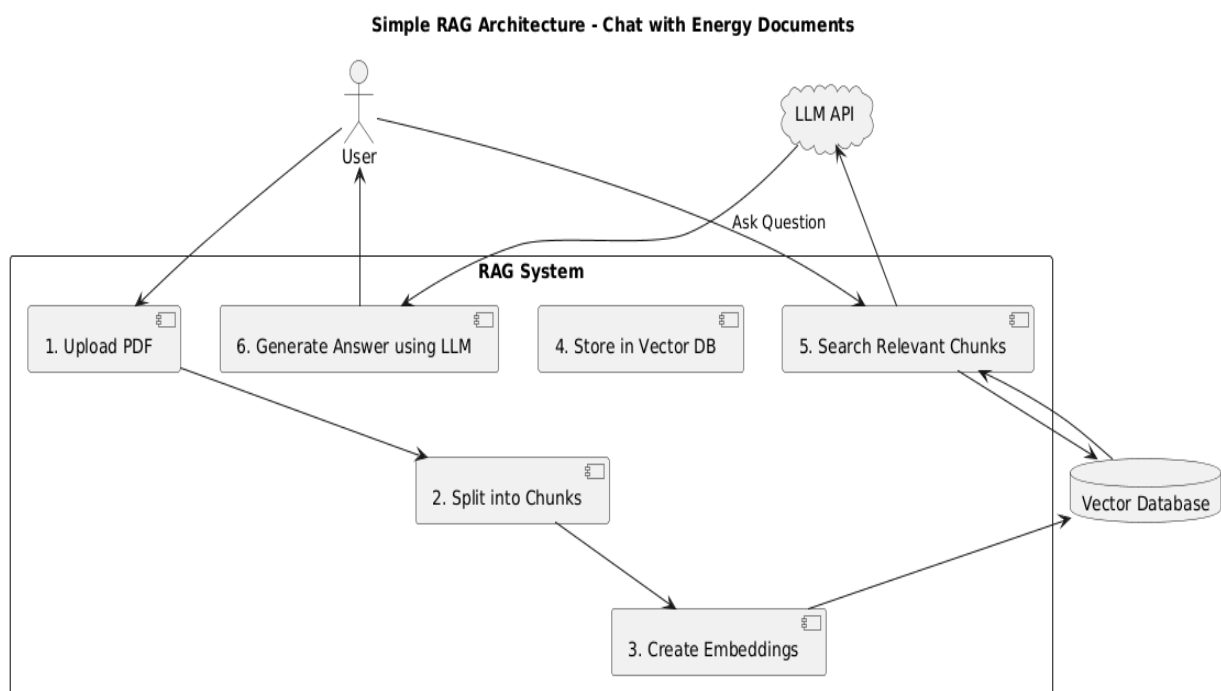


Fig.1: Simple RAG Architecture – Chat with Energy Document

2. System Design

2.1 Application Design

The application follows a modular layered architecture to ensure scalability, maintainability, and separation of concerns. The architecture consists of:

- Presentation Layer (API Endpoints)
- Service Layer (Business Logic)
- Repository Layer (Data Access)

- Data Layer (Vector DB + Metadata DB)

Each component is loosely coupled. For example, the embedding service can be replaced without modifying the retrieval logic. Similarly, the vector database can be replaced with a cloud-based alternative such as Pinecone without affecting higher layers.

This modular design ensures long-term extensibility.

2.2 Process Flow

The system supports two primary workflows: Document Upload Flow and Question-Answer Flow.

Document Upload Flow

When a user uploads a PDF document, the system first validates the file type and size. The document is stored securely and processed to extract text using a PDF parsing utility. The extracted text is cleaned and divided into overlapping chunks to preserve contextual continuity. Each chunk is converted into a numerical embedding. These embeddings, along with metadata such as document name and page number, are stored in the vector database and relational database.

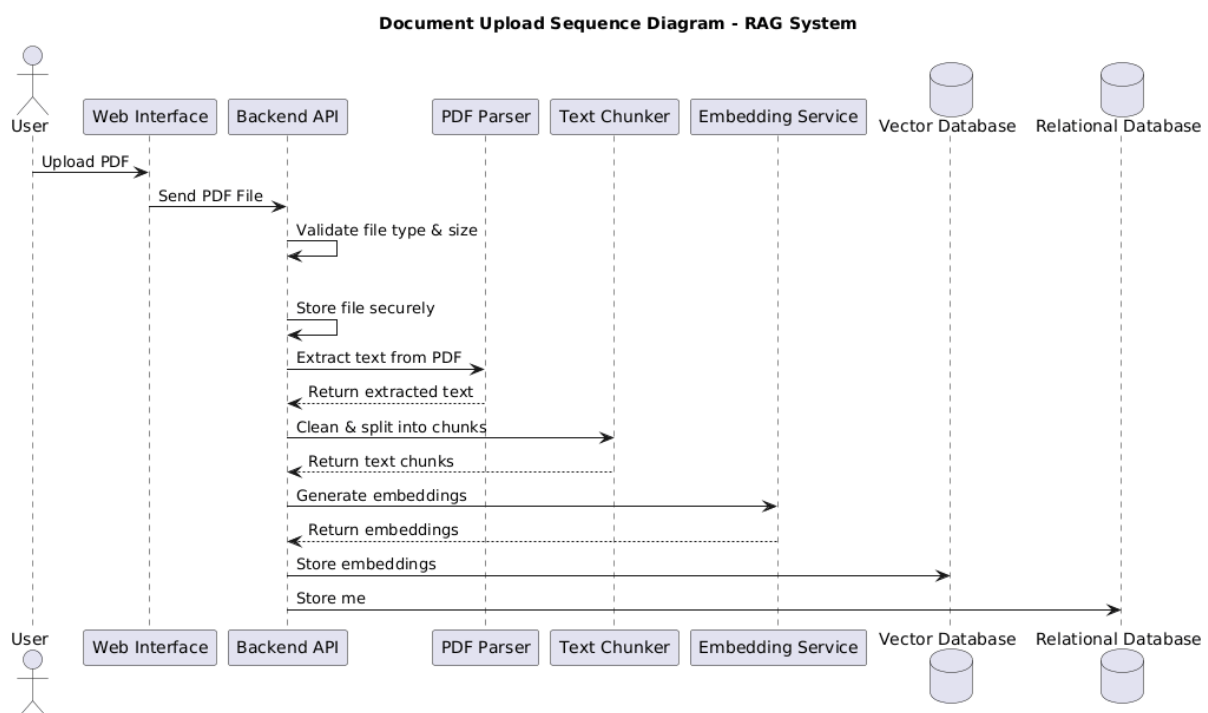


Fig 2: Document Upload Workflow

Question-Answer Flow

When a user submits a question, the system converts the query into an embedding using the same embedding model. A similarity search is performed in the vector database to retrieve the most relevant chunks. These retrieved chunks form the context passed to the LLM. The LLM generates a response strictly grounded in the retrieved content. If the information is not found, the system returns a predefined message.

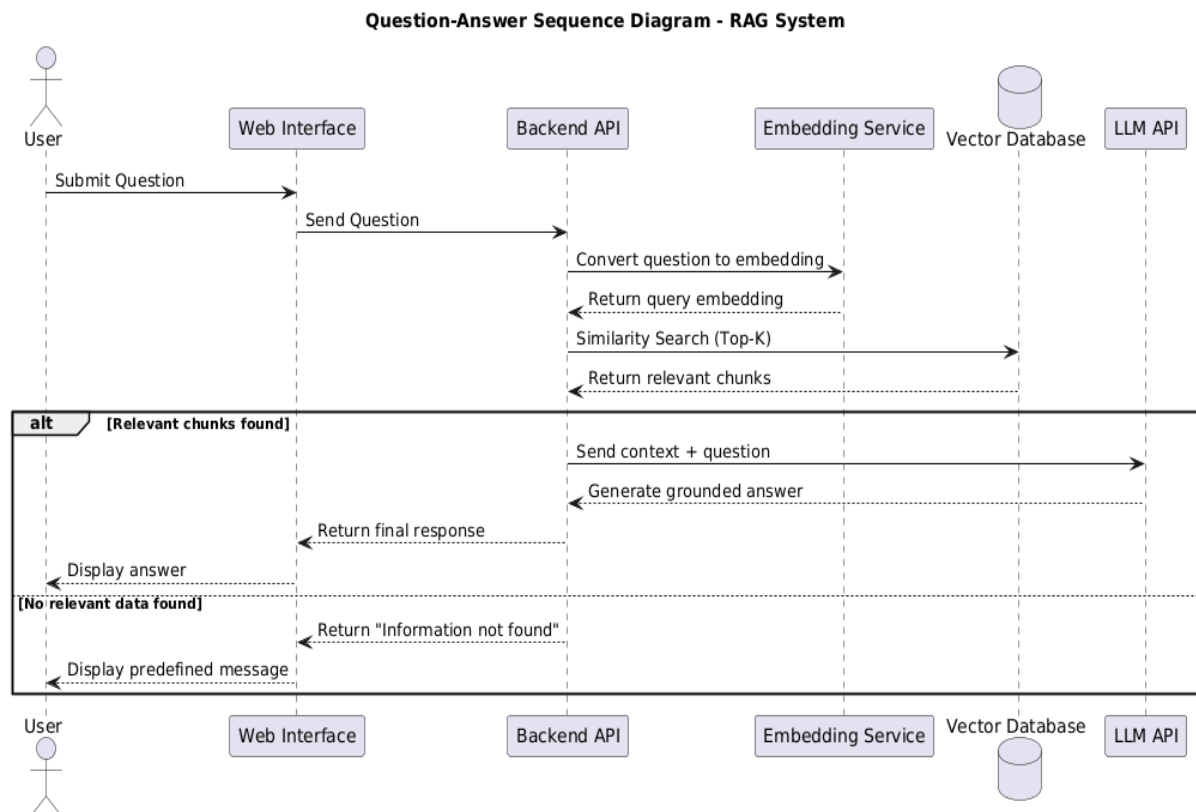


Fig 3: Question-Answer Flow

2.3 Information Flow

Information flows sequentially between modules in a structured pipeline. When a document is uploaded, data flows from the API layer to the document processor, then to the embedding service, and finally to the vector store and metadata database.

During question answering, the query flows from the API layer to the embedding service, then to the retriever module, and finally to the LLM service before returning a response to the user.

This controlled information flow ensures traceability, modularity, and consistent data transformation throughout the system lifecycle.

2.4 Components Design

The Document Processor component is responsible for extracting and cleaning text from PDF documents. It ensures that irrelevant formatting, whitespace issues, and special characters are handled properly before chunking the text.

The Embedding Service converts textual chunks into high-dimensional numerical vectors. These embeddings capture semantic meaning, enabling contextual similarity search rather than simple keyword matching.

The Vector Store component maintains the embedding index. It performs similarity search using cosine similarity and retrieves the most relevant chunks for a given query.

The Retriever component acts as a bridge between the embedding service and vector database. It ensures that only the top-k most relevant chunks are returned.

The LLM Service constructs a structured prompt containing retrieved context and user question. It ensures that the model is instructed to answer only from the provided context and respond appropriately when information is not available.

The API Layer manages endpoints for document upload, chat interaction, and authentication. It handles validation, request routing, and response formatting.

2.5 Key Design Considerations

One of the primary design considerations is grounding. The LLM is restricted to answer only from retrieved document content to prevent hallucination.

Another consideration is chunking strategy. Overlapping chunking ensures contextual continuity between segments, preventing information loss across paragraph boundaries.

Scalability is also considered by using modular architecture and pluggable vector databases. Performance optimization is achieved through batch embedding generation and efficient vector indexing.

Security and authentication mechanisms are integrated to prevent unauthorized access and misuse.

2.6 API Catalogue

The system exposes REST APIs through FastAPI.

The Upload API accepts multipart PDF files and returns confirmation along with a document identifier. This endpoint triggers the indexing pipeline.

The Chat API accepts a JSON request containing the user's question and returns a structured response including the generated answer and source references such as document name and page number.

Authentication APIs manage user registration and login, issuing JWT tokens for secured access.

3. Data Design

3.1 Data Model

The system maintains two primary forms of data storage: relational metadata storage and vector storage.

The metadata database stores document identifiers, document names, upload timestamps, page numbers, and chunk text. This ensures traceability and allows source citation in responses.

The vector database stores embeddings along with metadata references. Each embedding corresponds to a specific chunk of text and is indexed for efficient similarity search.

ER Diagram - RAG System Data Model

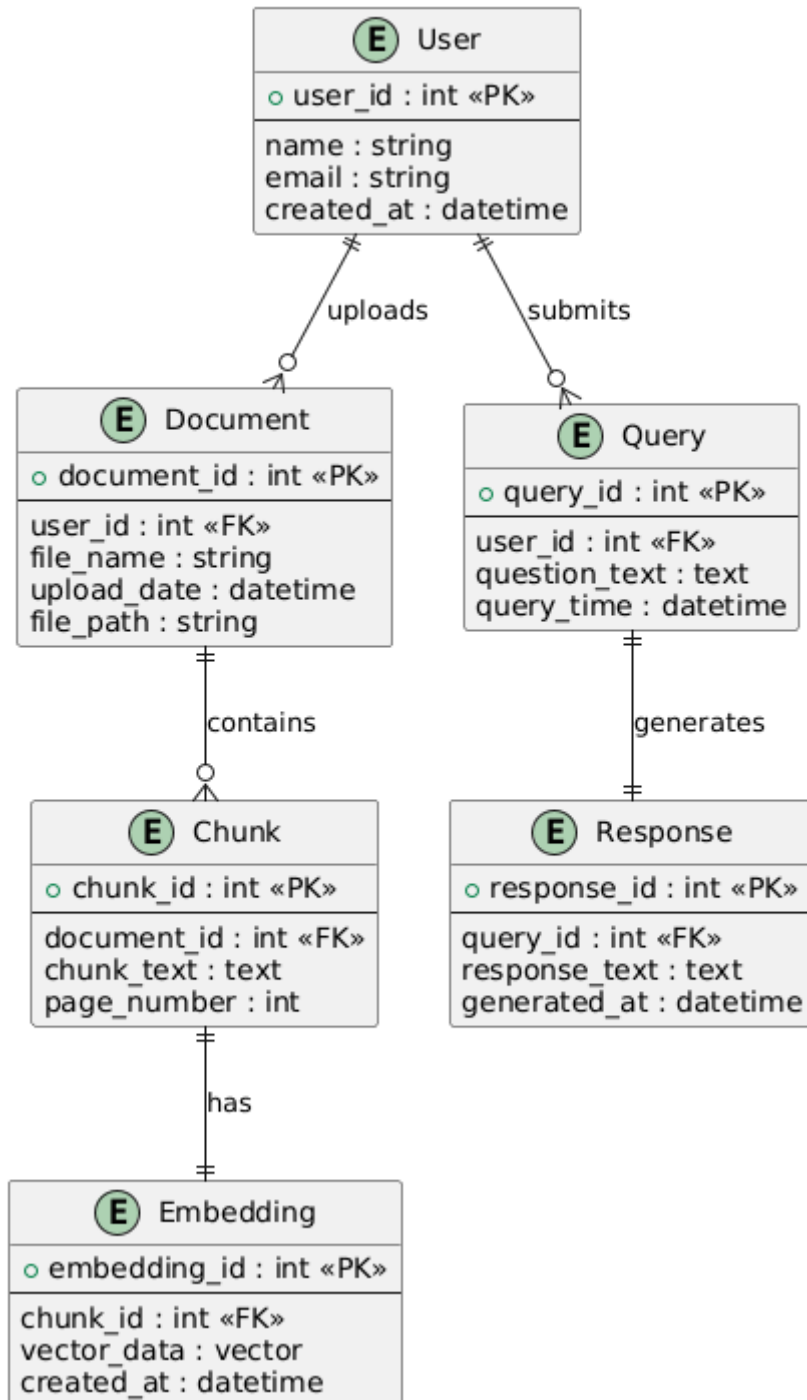


Fig 4: Metadata and Vector Storage Data Model

3.2 Data Access Mechanism

The system uses SQLAlchemy ORM for interacting with the relational metadata database. This ensures abstraction from raw SQL queries and improves maintainability.

For vector similarity search, FAISS is used to index and retrieve embeddings efficiently. The repository pattern abstracts vector storage, enabling future replacement with cloud-based vector databases such as Pinecone.

3.3 Data Retention Policies

Uploaded documents and their associated embeddings are retained until manually deleted by administrators. Logs related to system usage and API requests are maintained for a defined period to support monitoring and debugging.

Retention policies ensure compliance with data governance requirements while maintaining system transparency.

3.4 Data Migration

Database schema changes are managed using migration tools such as Alembic. In the event of embedding model updates, vector indices can be rebuilt to ensure compatibility. Backup and restore strategies are defined to prevent data loss.

4. Interfaces

The system interacts with external APIs for embeddings and LLM inference using OpenAI APIs. It exposes RESTful interfaces for client applications via FastAPI.

Internal interfaces between services are implemented using dependency injection, allowing seamless communication between modules without tight coupling.

5. State and Session Management

The system follows a stateless architecture for API interactions. Each request contains all necessary authentication information via JWT tokens. The server does not maintain session state between requests, which improves scalability and simplifies horizontal scaling.

JWT Authentication Flow Diagram

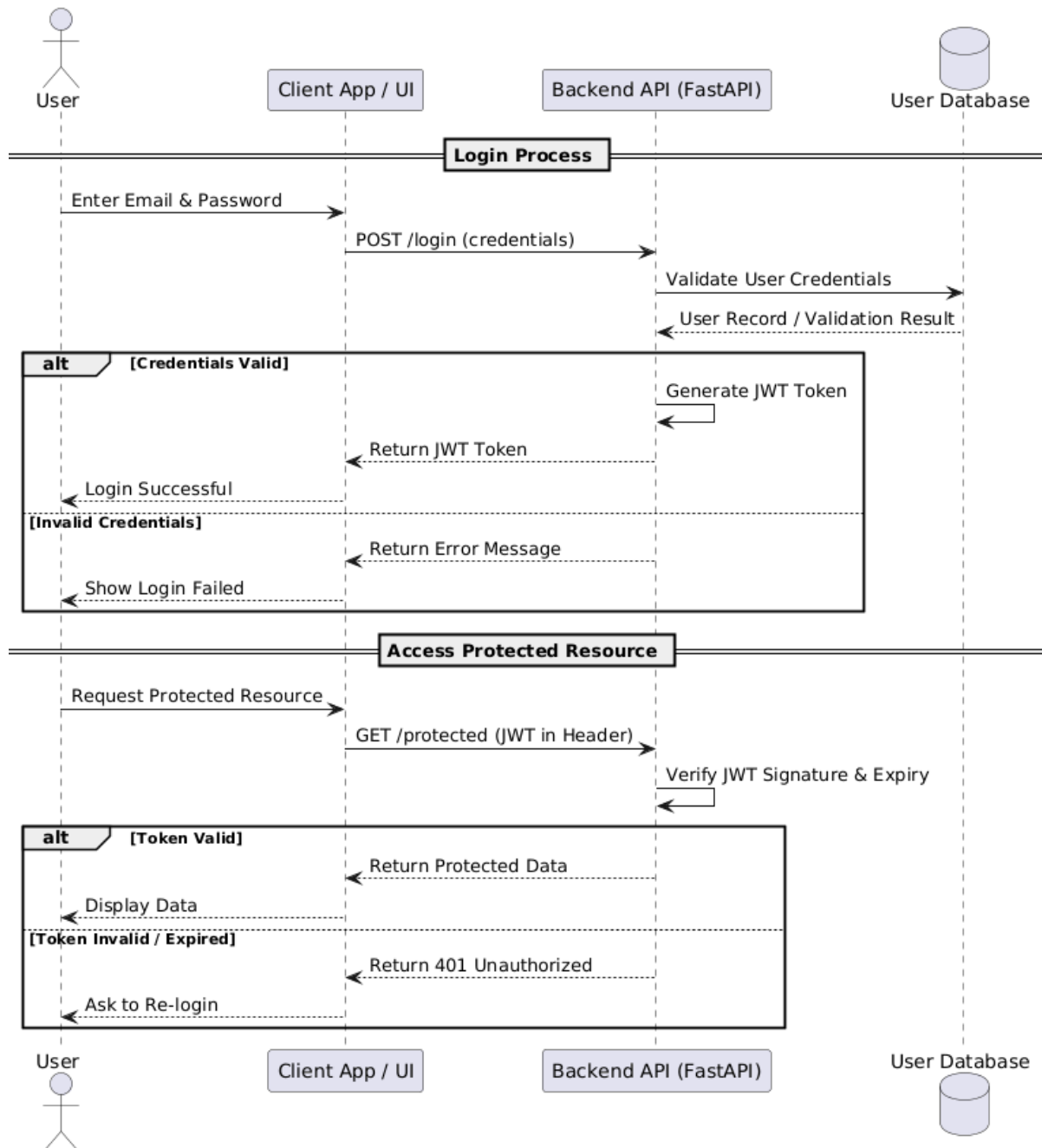


Fig 5: JWT-Based Authentication and Protected diagram

6. Caching

Caching strategies are implemented to improve performance. The FAISS index is loaded into memory during application startup to avoid repeated disk reads. In future enhancements, Redis can be introduced to cache frequently asked questions and repeated query responses.

Caching reduces latency and enhances user experience.

7. Non-Functional Requirements

7.1 Security Aspects

Security measures include JWT-based authentication, file validation, API key protection, rate limiting, and CORS enforcement. The system ensures that uploaded files are validated before processing. Prompt guardrails restrict LLM responses to retrieved document context, minimizing hallucination risks.

7.2 Performance Aspects

The system is designed to handle multiple document uploads and concurrent queries efficiently. Batch embedding generation reduces API overhead. Asynchronous endpoints improve request handling. The vector search mechanism ensures low-latency retrieval, targeting average response times under three seconds.

8. References

- [1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [2] OpenAI, "OpenAI API Documentation," 2024. [Online]. Available: <https://platform.openai.com/docs>
- [3] J. Johnson, M. Douze, and H. Jégou, "FAISS: A Library for Efficient Similarity Search and Clustering of Dense Vectors," Facebook AI Research, 2017.
- [4] FastAPI, "FastAPI Documentation," 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
- [5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, 2015.
- [6] Pinecone Systems Inc., "Pinecone Vector Database Documentation," 2024. [Online]. Available: <https://docs.pinecone.io/>